

Proyecto: CustomDrive CR

Integrante/equipo: Hugo Gurdíán y equipo

Adaptación técnica: aunque la guía menciona JSP, Servlets, DAO y JDBC, el proyecto se implementó con Spring Boot, Thymeleaf, Spring MVC, Spring Data JPA y MySQL, manteniendo una arquitectura MVC/por capas equivalente.

Universidad de Costa Rica
Desarrollo de Software III
Proyecto Final Web

Práctica Guiada 2

Arquitectura MVC, conexión a MySQL, autenticación y primer CRUD funcional

1. Datos generales

Elemento	Descripción
Curso	Desarrollo de Software III
Tipo de actividad	Práctica guiada de avance del proyecto final
Modalidad	Trabajo grupal
Duración sugerida	3 horas
Tecnologías base	Java, JSP, Servlets, JDBC, MySQL, HTML5, CSS3 y Bootstrap o estilos propios
Producto esperado	Proyecto Java Web con estructura MVC inicial, conexión JDBC, login funcional, control de sesión, roles y primer CRUD conectado a MySQL

2. Propósito de la práctica

Esta práctica tiene como propósito que cada equipo convierta el diseño técnico elaborado en la primera práctica en una base funcional de aplicación web. La prioridad es que el proyecto deje de ser únicamente una propuesta visual y cuente con arquitectura organizada, conexión real a MySQL, inicio de sesión, protección de páginas y un primer módulo CRUD completo.

Al finalizar, cada equipo deberá demostrar que el sistema puede ejecutarse en un servidor de aplicaciones, conectarse a la base de datos, autenticar usuarios y administrar al menos una entidad principal del proyecto.

3. Objetivo general

Implementar la base funcional del proyecto web mediante una arquitectura MVC o limpia, integrando conexión a MySQL, autenticación, manejo de sesiones, control de roles y el primer CRUD completo del sistema.

4. Objetivos específicos

1. Crear la estructura inicial del proyecto bajo una arquitectura organizada por capas.
2. Configurar la conexión a MySQL mediante JDBC y consultas parametrizadas.
3. Implementar autenticación de usuarios con validación contra la base de datos.
4. Aplicar manejo de sesiones utilizando HttpSession.
5. Proteger páginas privadas y controlar el acceso según el rol del usuario.
6. Implementar el primer módulo CRUD completo conectado a MySQL.
7. Registrar evidencias técnicas de funcionamiento para la documentación final.

5. Contexto de trabajo

En la práctica anterior, cada equipo definió el problema, usuarios, roles, módulos, requerimientos y base de datos preliminar. En esta segunda práctica, ese diseño se lleva a código. El trabajo debe respetar la separación entre presentación, controladores, lógica de negocio y acceso a datos.

No se permite resolver el sistema colocando toda la lógica dentro de páginas JSP. Las páginas JSP deben utilizarse principalmente para mostrar información y formularios; los Servlets deben recibir y procesar solicitudes; los DAO deben concentrar el acceso a la base de datos; y los modelos deben representar las entidades del sistema.

6. Actividad 1: Preparación del entorno de desarrollo

Cada equipo debe verificar que el entorno esté listo antes de programar. La práctica no debe avanzar si el proyecto no puede ejecutarse localmente o si no existe una base de datos disponible para pruebas.

Checklist mínimo

- IDE configurado: NetBeans, IntelliJ IDEA, Eclipse o Visual Studio Code con soporte para Java Web.
- Servidor de aplicaciones disponible: Apache Tomcat, GlassFish u otro compatible.
- MySQL instalado o disponible mediante un servidor local/remoto.
- Driver JDBC de MySQL agregado al proyecto.
- Repositorio Git actualizado y compartido con todos los integrantes.
- Carpeta database con schema.sql y data.sql actualizados.
- README inicial con instrucciones para ejecutar el proyecto.

Producto de la actividad

Elemento revisado	Estado	Evidencia o comentario
Servidor de aplicaciones	Sí	Proyecto ejecutándose localmente en Spring Boot con servidor embebido Apache Tomcat en http://localhost:8080 .
Base de datos MySQL	Sí	Base de datos MySQL customdrivecr conectada correctamente;

Elemento revisado	Estado	Evidencia o comentario
		Hibernate/JPA genera y actualiza tablas.
Driver JDBC	Sí	Driver MySQL agregado desde la dependencia MySQL Driver del proyecto Maven Spring Boot.
Repositorio Git	Sí	Repositorio creado y actualizado en GitHub con los últimos cambios del proyecto, README.md, carpeta database y documentación correspondiente.
Scripts SQL	Sí	Script SQL final agregado en la carpeta database del repositorio. Incluye creación del esquema y datos de prueba para ejecutar el sistema.
README inicial	Sí	README.md agregado al repositorio con descripción del proyecto, tecnologías, rutas principales, configuración de base de datos e instrucciones de ejecución.

7. Actividad 2: Creación de la estructura MVC del proyecto

Cada equipo debe organizar el código del proyecto siguiendo una estructura clara. La organización puede variar según el IDE, pero debe evidenciar separación de responsabilidades.

Estructura sugerida

```

nombre-proyecto/
|-- src/main/java/
|   |-- model/      Entidades del sistema
|   |-- dao/        Acceso a datos mediante JDBC
|   |-- service/     Reglas de negocio o validaciones de aplicación
|   |-- servlet/     Controladores web
|   |-- util/        Conexión, constantes y utilidades
|   |-- filter/      Filtros de seguridad, si se utilizan
|-- src/main/webapp/
|   |-- views/       Páginas JSP privadas
|   |-- auth/        JSP de login o recuperación
|   |-- css/         Estilos
|   |-- js/          Scripts
|   |-- index.jsp
|-- database/
|   |-- schema.sql
|   |-- data.sql
|-- README.md

```

Producto de la actividad

Capa o paquete	Responsabilidad	Clase o archivo creado
model	Representa las entidades principales del sistema.	Paquete entity: Usuario, Marca, Modelo, Vehiculo, Accesorio, Compatibilidad, Pedido, DetallePedido.
dao	Contiene consultas SQL y operaciones contra MySQL.	Paquete repository: UsuarioRepository, MarcaRepository, ModeloRepository, VehiculoRepository, AccesorioRepository, CompatibilidadRepository, PedidoRepository, DetallePedidoRepository. Equivale al acceso a datos; usa Spring Data JPA sobre MySQL.
service	Contiene reglas de negocio, validaciones o coordinación entre DAO.	Paquete service: UsuarioService, MarcaService, ModeloService, VehiculoService, AccesorioService, CompatibilidadService, PedidoService.
servlet	Recibe solicitudes HTTP, procesa acciones y redirige respuestas.	Paquete controller: AuthController, ClienteController, CarritoController, PedidoController, AdminDashboardController, AdminUsuarioController, AdminAccesorioController, AdminCompatibilidadController, AdminPedidoController, AdminReporteController.
views / JSP	Muestra formularios, tablas y mensajes al usuario.	Carpeta templates: login.html, seleccion-vehiculo.html, catalogo.html, detalle-producto.html, carrito.html, confirmar-pedido.html, factura.html y vistas admin.
util	Contiene conexión JDBC y utilidades generales.	Paquete config: AdminInterceptor y WebConfig para protección de rutas /admin/**.

8. Actividad 3: Configuración de conexión a MySQL con JDBC

Cada equipo debe crear una clase responsable de abrir la conexión a la base de datos. Esta clase debe estar en un paquete de utilidad y debe reutilizarse desde los DAO. No se deben abrir conexiones directamente desde las páginas JSP.

Ejemplo base de clase de conexión

```
package util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

```
public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/nombre_bd";
    private static final String USER = "root";
    private static final String PASSWORD = "";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

El equipo debe adaptar el nombre de la base de datos, usuario y contraseña según su entorno. Se recomienda evitar colocar credenciales reales en repositorios públicos.

Producto de la actividad

- Clase DBConnection o equivalente creada.
- Prueba de conexión ejecutada correctamente.
- Captura de pantalla o evidencia del servidor ejecutando el proyecto.

Nota de adaptación: en CustomDrive CR la conexión a MySQL se realiza mediante application.properties, MySQL Driver, Spring Data JPA e Hibernate. Los repositorios JPA cumplen el rol de acceso a datos equivalente a DAO/JDBC en la guía.

- Confirmación de que el proyecto se conecta a la base de datos definida en schema.sql.

9. Actividad 4: Implementación de autenticación y sesiones

Cada equipo debe implementar un inicio de sesión funcional. Las credenciales deben validarse contra la tabla de usuarios en MySQL. El sistema debe crear una sesión cuando el usuario inicia correctamente y destruirla cuando cierra sesión.

Elementos mínimos del login

- Página login.jsp o equivalente con formulario de correo y contraseña.
- Servlet LoginServlet que reciba la solicitud POST.
- UsuarioDAO con método para buscar usuario por credenciales.
- Uso de HttpSession para guardar usuario autenticado y rol.
- Servlet LogoutServlet para cerrar sesión.
- Redirección a una página privada cuando el login sea correcto.
- Mensaje de error cuando las credenciales sean inválidas.

Flujo esperado

```

Usuario ingresa correo y contraseña
|
v
LoginServlet recibe la solicitud POST
|
v
UsuarioDAO consulta MySQL con PreparedStatement
|
v
Si existe el usuario: crear HttpSession y redirigir al panel
Si no existe: volver al login con mensaje de error
```

Producto de la actividad

Elemento	Estado	Evidencia
Formulario de login	Sí	login.html con formulario POST a /login, campos correo y password, y mensaje de error con Thymeleaf.
LoginServlet	Sí, equivalente Spring MVC	AuthController recibe GET/POST /login, valida credenciales y redirige según rol.
UsuarioDAO	Sí, equivalente Repository	UsuarioRepository consulta MySQL con findByCorreoAndPasswordAndActivoTrue.
HttpSession	Sí	HttpSession guarda usuarioLogueado, rolUsuario y nombreUsuario.
LogoutServlet	Sí, equivalente Controller	/logout invalida la sesión y redirige al login.
Mensajes de error	Sí	Cuando las credenciales son inválidas se muestra: Correo o contraseña incorrectos.

10. Actividad 5: Protección de páginas privadas y control de roles

El sistema debe impedir que un usuario no autenticado acceda directamente a páginas privadas copiando la URL en el navegador. Además, los roles definidos por el equipo deben tener accesos diferenciados según sus responsabilidades.

Acciones requeridas

- Crear una validación de sesión en cada Servlet privado o implementar un filtro de autenticación.
- Redirigir al login cuando no exista una sesión activa.
- Ocultar o deshabilitar opciones de menú que no correspondan al rol del usuario.
- Validar permisos también en el backend, no solo en la interfaz.
- Probar al menos dos usuarios con roles diferentes.

Matriz de acceso

Módulo o página	Administrador	Usuario regular	Observaciones
Panel principal	Sí	No	/admin/dashboard protegido para ADMIN; CLIENTE es redirigido a /seleccion-vehiculo.
Gestión de usuarios	Sí	No	/admin/usuarios solo accesible para ADMIN mediante AdminInterceptor.
Primer CRUD del proyecto	Sí	No	CRUD de accesorios, marcas, modelos, vehículos y compatibilidades solo para

Módulo o página	Administrador	Usuario regular	Observaciones
			ADMIN.
Reportes o consultas	Sí	No	/admin/reporte-ventas y /admin/pedidos protegidos para ADMIN.
Configuración o mantenimiento	Sí	No	Módulos administrativos protegidos por sesión y rol.

11. Actividad 6: Implementación del primer CRUD funcional

Cada equipo debe seleccionar una entidad principal del proyecto e implementar un CRUD completo. Se recomienda iniciar con una entidad central, pero no excesivamente compleja. Por ejemplo: productos, clientes, libros, citas, solicitudes, cursos, eventos, empleados, recursos o incidencias.

Operaciones mínimas del CRUD

- Registrar un nuevo elemento.
- Listar los elementos existentes.
- Consultar el detalle de un elemento.
- Editar un elemento.
- Eliminar o desactivar un elemento.
- Mostrar mensajes de éxito, advertencia o error.
- Validar campos obligatorios desde el servidor.
- Utilizar PreparedStatement para las consultas SQL.

Estructura recomendada para el CRUD

Componente	Responsabilidad	Ejemplo
Modelo	Representa la entidad del sistema.	Producto.java
DAO	Ejecuta INSERT, SELECT, UPDATE y DELETE o desactivación.	ProductoDAO.java
Servlet	Procesa las acciones del usuario.	ProductoServlet.java
JSP listar	Muestra una tabla con los registros.	productos/listar.jsp
JSP formulario	Permite registrar o editar datos.	productos/formulario.jsp
JSP detalle	Muestra la información completa de un registro.	productos/detalle.jsp

Producto de la actividad

Operación	Ruta o archivo	Funciona correctamente	Observaciones
Registrar	/admin/accesorios/guardar	Sí	El formulario permite registrar nombre, categoría, descripción, precio, stock,

Operación	Ruta o archivo	Funciona correctamente	Observaciones
			imagenUrl y activo.
Listar	/admin/accesorios	Sí	Lista todos los accesorios registrados desde MySQL.
Consultar detalle	/detalle-producto?id={id}	Sí	La vista cliente muestra el detalle real del accesorio seleccionado.
Editar	/admin/accesorios/editar/{id}	Sí	Carga el accesorio en el formulario y permite actualizar sus datos.
Eliminar o desactivar	/admin/accesorios/eliminar/{id}	Sí	Elimina el accesorio seleccionado desde el módulo administrativo.
Validaciones	Formulario admin/accesorios.html y entidad Accesorio	Sí	El sistema cuenta con validaciones en formularios, control visual de stock, bloqueo de productos sin stock y mensajes claros para el usuario.

12. Actividad 7: Validaciones, mensajes y manejo básico de errores

El primer CRUD debe incluir validaciones mínimas. No se aceptará como funcional un formulario que permita guardar datos vacíos, inconsistentes o con errores evidentes. Las validaciones deben realizarse en el servidor, aunque también pueden reforzarse con JavaScript o atributos HTML5.

Validaciones sugeridas

Tipo de validación	Ejemplo	Lugar mínimo requerido
Campo obligatorio	Nombre, descripción, fecha, correo o estado.	Servidor
Formato	Correo electrónico, teléfono o fecha válida.	Servidor y cliente
Longitud	Evitar textos demasiado cortos o demasiado largos.	Servidor
Integridad	No registrar una relación con un ID inexistente.	Servidor
Duplicados	Evitar correos, códigos o nombres únicos repetidos.	Servidor

Producto de la actividad

- Mensajes de éxito al guardar, editar o eliminar.
- Mensajes de error cuando falten datos obligatorios.
- Manejo básico de excepciones en DAO o Servlet.
- Validación de datos antes de ejecutar INSERT o UPDATE.
- Evitar mostrar errores técnicos directamente al usuario final.

13. Actividad 8: Prueba guiada del avance

Cada equipo debe ejecutar una prueba corta del avance para verificar que la base funcional del sistema está lista para continuar con los siguientes módulos.

Caso de prueba	Resultado esperado	Resultado obtenido
Ingresar con usuario válido	El sistema crea sesión y muestra el panel principal.	Correcto. Admin accede al panel y cliente accede a selección de vehículo.
Ingresar con usuario inválido	El sistema muestra mensaje de error.	Correcto. Se muestra mensaje de error en login.html.
Acceder a página privada sin sesión	El sistema redirige al login.	Correcto. AdminInterceptor redirige a /login cuando no existe sesión.
Acceder con rol restringido	El sistema limita opciones o bloquea el acceso.	Correcto. Usuario CLIENTE no puede entrar a /admin/** y es redirigido a /seleccion-vehiculo.
Registrar dato en el CRUD	El dato se guarda en MySQL.	Correcto. Accesorios, marcas, modelos, vehículos, compatibilidades, usuarios y pedidos guardan en MySQL.
Editar dato en el CRUD	El dato se actualiza correctamente.	Correcto. Editar accesorio y demás CRUD actualiza datos correctamente.
Eliminar o desactivar dato	El dato se elimina o cambia de estado.	Correcto. Eliminar/desactivar funciona según el módulo; usuarios pueden desactivarse.
Enviar formulario incompleto	El sistema muestra validación.	Sí. El sistema cuenta con validaciones HTML5, restricciones básicas en formularios, control de stock y mensajes para orientar al usuario.

14. Entregables de la práctica

Al finalizar la práctica, cada equipo debe subir al repositorio o entregar las siguientes evidencias:

1. Proyecto Java Web ejecutándose correctamente en el servidor de aplicaciones.
2. Estructura MVC o limpia creada en el proyecto.
3. Clase de conexión JDBC funcionando.
4. Login funcional conectado a MySQL.

5. Manejo de sesión con HttpSession.
6. Cierre de sesión funcional.
7. Control básico de roles.
8. Primer CRUD completo conectado a MySQL.
9. Validaciones básicas del primer CRUD.
10. Capturas de pantalla del avance.
11. README actualizado con instrucciones de ejecución.
12. Scripts schema.sql y data.sql actualizados.

15. Criterios de evaluación

Criterio	Puntaje
Configura correctamente el entorno, servidor, base de datos y repositorio.	10 pts
Organiza el proyecto bajo una arquitectura MVC o limpia.	15 pts
Implementa conexión JDBC funcional y reutilizable.	15 pts
Implementa login, logout y manejo de sesión.	15 pts
Aplica control básico de roles y protección de páginas privadas.	10 pts
Desarrolla un primer CRUD completo conectado a MySQL.	25 pts
Incluye validaciones, mensajes y manejo básico de errores.	5 pts
Actualiza README, scripts SQL y evidencias del avance.	5 pts
Total	100 pts

16. Preguntas de cierre

Cada equipo debe responder brevemente las siguientes preguntas al finalizar la práctica:

1. ¿Cuál fue la mayor dificultad técnica durante la implementación?
2. ¿Qué parte del proyecto ya está realmente conectada a la base de datos?
3. ¿El login valida credenciales reales desde MySQL?
4. ¿Qué rol tiene más permisos y qué rol tiene menos permisos?
5. ¿Cuál fue el primer CRUD implementado y por qué se eligió?
6. ¿Qué validaciones faltan por mejorar?
7. ¿Qué módulo se implementará en la tercera práctica guiada?

Respuestas de cierre - CustomDrive CR

1. ¿Cuál fue la mayor dificultad técnica durante la implementación?

Integrar correctamente las relaciones entre vehículos, accesorios y compatibilidades sin romper el flujo del catálogo, además de proteger rutas administrativas con sesión y roles.

2. ¿Qué parte del proyecto ya está realmente conectada a la base de datos?

Login de usuarios, CRUDs administrativos, compatibilidades, carrito convertido en pedido, detalle de pedido, comprobante y reporte de ventas están conectados a MySQL.

3. ¿El login valida credenciales reales desde MySQL?

Sí. AuthController usa UsuarioService y UsuarioRepository para validar correo, contraseña y estado activo contra la tabla usuarios.

4. ¿Qué rol tiene más permisos y qué rol tiene menos permisos?

ADMIN tiene más permisos porque puede ingresar al panel administrativo y gestionar catálogos, usuarios, pedidos y reportes. CLIENTE tiene menos permisos y solo accede al flujo público/cliente de selección de vehículo, catálogo, carrito y compra.

5. ¿Cuál fue el primer CRUD implementado y por qué se eligió?

El primer CRUD implementado fue Accesorios, porque representa el inventario principal del sistema y es la entidad central para mostrar productos compatibles al cliente.

6. ¿Qué validaciones faltan por mejorar?

Las validaciones principales ya fueron aplicadas: campos obligatorios, valores mínimos, control visual de stock, bloqueo de productos sin stock y mensajes de apoyo para el usuario. Como mejora futura podría reforzarse aún más la validación del lado servidor.

7. ¿Qué módulo se implementará en la tercera práctica guiada?

Se propone continuar con el pulido de diseño, validaciones avanzadas, scripts SQL finales, README, evidencias, documentación y mejoras de reportes.

17. Resultado esperado

Al finalizar esta práctica, cada equipo debe contar con la base técnica funcional del proyecto final. El sistema debe ejecutarse, conectarse a MySQL, permitir inicio y cierre de sesión, proteger páginas privadas, diferenciar al menos dos roles y administrar una entidad mediante un CRUD completo.

Esta práctica servirá como base para la siguiente etapa, en la cual se completarán los CRUD restantes, se incorporarán reportes o consultas, se reforzarán validaciones, se documentarán pruebas y se preparará la defensa final del proyecto.

18. Checklist rápido antes de salir del laboratorio

Verificación final	Sí / No	Comentario
El proyecto compila y despliega correctamente.	Sí	Compila y ejecuta localmente en Spring Boot con Tomcat embebido.
La base de datos se crea con	Sí	Se agregó script SQL final en la

Verificación final	Sí / No	Comentario
schema.sql.		carpeta database del repositorio para crear la base customdrivecr y sus tablas.
data.sql contiene usuarios de prueba.	Sí	El script SQL final incluye datos de prueba, usuarios, marcas, modelos, vehículos, accesorios y compatibilidades necesarias para la demostración.
El login funciona con datos reales.	Sí	Login valida credenciales reales desde tabla usuarios en MySQL.
Las páginas privadas no se abren sin sesión.	Sí	/admin/** redirige a login sin sesión.
Existe al menos un usuario administrador y uno regular.	Sí	ADMIN y CLIENTE creados y probados.
El primer CRUD guarda, lista, edita y elimina/desactiva.	Sí	CRUD de accesorios funciona; además existen CRUDs de marcas, modelos, vehículos, compatibilidades y usuarios.
El código no tiene SQL dentro de los JSP.	Sí	Las vistas Thymeleaf solo muestran datos y formularios; la lógica está en controller, service y repository.
El repositorio tiene commits recientes del equipo.	Sí	Repositorio GitHub actualizado con código fuente, README.md, base de datos y documentación del proyecto.